Part-2: Implementation

Student Name: Samarth Gunvantbhai Desai

SRN: 21079043

Module Leader: John Kanyaru

Module Name: Software Architecture

Module Code: 6COM2013

Reflective report

The Parcel Depot Management System is designed and implemented following the models developed in Part 1. The project clearly understands object-oriented programming principles and employs the Model-View-Controller (MVC) pattern to ensure separation of concerns, scalability, and maintainability.

Realization of Models

The implementation aligns closely with the conceptual models developed in part 1:

1. **Use Case Implementation:**
   - **Customer Management:** The functionality of managing customers is implemented in the `QueueofCustomers` class, which maintains a queue for efficient processing. The method `addCustomer` adds customers to the queue, while `removeCustomer` processes the next customer in line. This implementation reflects the model developed for queue-based customer handling.
   - **Parcel Management:** Parcel-related operations are handled by the `ParcelMap` class. The methods `addParcel` and `updateParcelStatus` allow for adding new parcels to the depot and updating their status, respectively. These operations correspond directly to the parcel management use case.
2. **Class Relationships:**
   - The `Manager` class acts as the central coordinator, tying together the `QueueofCustomers`, `ParcelMap`, and `Worker` classes. This relationship reflects the high-level class diagram, where `Manager` orchestrates operations between models.
   - The `Worker` class encapsulates business logic, such as calculating parcel collection fees and processing customers. For example, the `processCustomer` method in `Worker` updates the parcel's status and calculates the collection fee, directly mapping to the operational requirements.
3. **Report Generation:** The `Log` class is responsible for maintaining event logs and generating comprehensive reports. Methods such as `addCollectedParcel`, `addAddedCustomer`, and `writeReport` ensure that all activities in the depot are tracked and documented. This aligns with the reporting functionality described in the initial models.

Use of the MVC Pattern

The project follows the MVC design pattern to provide a clear separation of concerns:

- **Model:**
  - The `Customer`, `Parcel`, `QueueofCustomers`, and `ParcelMap` classes represent the core data and business logic. These classes ensure that the system's state is maintained and manipulated effectively.
  - The `Log` class serves as a utility for tracking system events and generating reports.
- **View:**
  - The `MainFrame` class implements the user interface. It provides a clear and interactive GUI for users to perform actions such as adding customers and parcels, processing customers, and generating reports. Features like alternating row colours, colour-coded parcel statuses, and an interactive summary enhance user engagement.
- **Controller:**
  - The `Manager` class acts as the controller, bridging the model and view. It handles user requests (e.g., adding customers or parcels) and updates the model and view accordingly.

Challenges and Solutions

One challenge was ensuring that the report accurately reflects added customers and parcels. This was resolved by implementing and refining methods in the `Log` class, such as `addAddedCustomer` and `addAddedParcel`, to track these activities. Another challenge was maintaining GUI responsiveness and ensuring alignment with the data model. This was achieved through the effective use of table models and renderers in the `MainFrame` class.

Reflection

The use of the MVC pattern significantly enhanced the system's structure. By decoupling data management, business logic, and presentation, the application is easier to test, debug, and extend. For example, adding new features, such as interactive summaries and enhanced visual indicators, was straightforward without affecting core functionality.

In summary, the project successfully realizes the models developed in part 1 through a robust implementation in part 2. The system is modular, user-friendly, and scalable, reflecting the principles of good software engineering practice.